Project - High Level Design

on

Dockerized Entertainment React Service

Course Name: Devops

Institution Name: Medicaps University

Course: Datagami Skill Based Course

Student Name(s) & Enrollment Number(s):

Sr no	Student Name	Enrolment Number
1	PRABAL SHUKLA	EN22CS301713
2	PRATIK KOUSHAL	EN22CS301747
3	PRIYANSH CHOUHAN	EN22CS301759
4	PUSHPRAJ SINGH CHOUHAN	EN22CS301774
5	RAJAT MALVIYA	EN22CS301783

Group : 11D7

Project no:DO-34

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **Dockerized Entertainment React Service** is a web-based application developed to demonstrate modern DevOps practices using containerized application deployment. The system provides a simple entertainment interface where users can access movie-related information through a web browser.

The application is developed using **React.js** as the frontend framework and is packaged inside a **Docker container** to ensure consistent behavior across different environments. Containerization ensures portability, scalability, and ease of deployment.

The system architecture is designed to support modern DevOps workflows including automated builds, container-based deployments, and scalable cloud infrastructure. The project also demonstrates how applications can be integrated with CI/CD pipelines and deployed to cloud environments such as AWS.

The goal of the system is to provide a simple yet effective example of how modern web applications can be developed, containerized, and deployed using DevOps principles.

1.1 Scope of the Document

This High Level Design (HLD) document describes the overall architecture and system design of the **Dockerized Entertainment React Service**.

The document includes:

- Overview of system architecture
- Application design and process flow
- Information flow between system components
- DevOps pipeline architecture
- Data model design
- System interfaces and deployment architecture

The purpose of this document is to provide a clear understanding of how different components of the system interact with each other and how the application is deployed in a containerized environment.

1.2 Intended Audience

This document is intended for the following stakeholders:

Developers – To understand the high-level structure of the application and system architecture.

DevOps Engineers – To understand containerization, CI/CD pipelines, and deployment strategies.

Project Evaluators / Faculty – To evaluate the architecture, design decisions, and implementation strategy of the project.

Future Contributors – To help extend the system with additional features such as backend services, databases, and scalable cloud deployment.

1.3 System Overview

The system is designed as a containerized web application where the frontend interface is developed using React and deployed using Docker containers.

The user accesses the application through a web browser. The browser sends an HTTP request to the Docker container hosting the application. The container runs the React application which processes the request and returns the user interface to the browser.

This architecture provides several advantages such as portability, scalability, and maintainability. Since the application runs inside Docker containers, it can be easily deployed on any system with Docker installed.

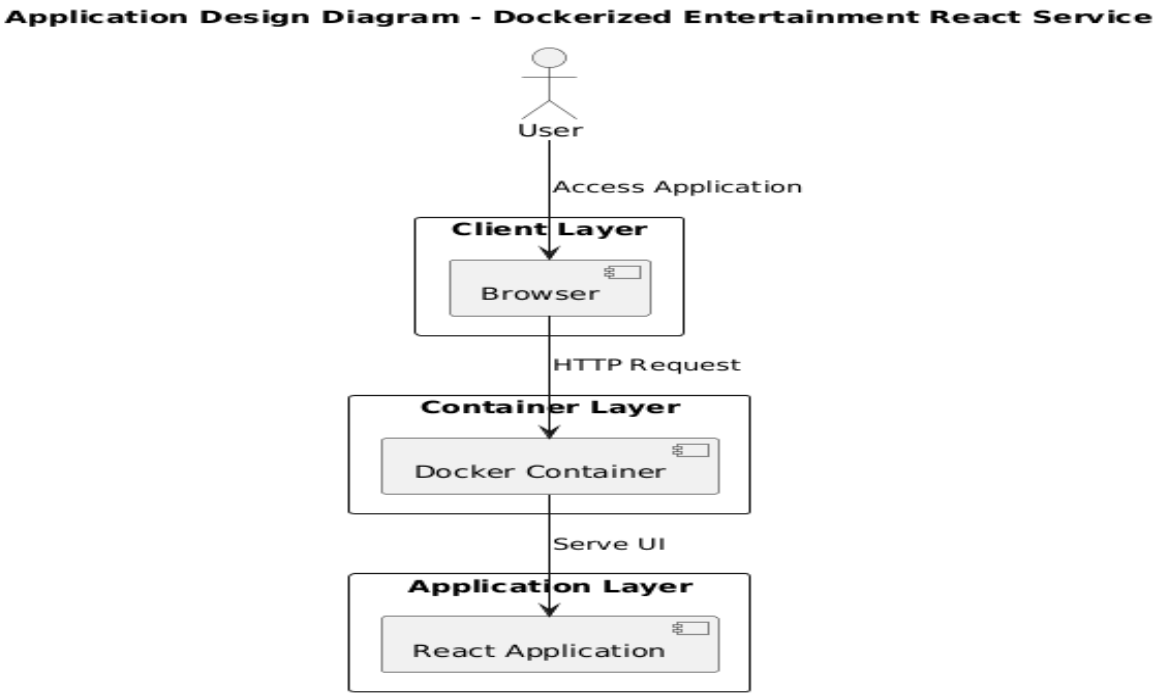

Figure 1: Application Architecture Diagram.

2. System Design

2.1 Application Design

The application follows a **layered architecture model** that separates different responsibilities of the system.

The architecture consists of the following layers:

Client Layer
This layer represents the user interface through which users interact with the application. Users access the system through a web browser.

Container Layer
The Docker container acts as the execution environment where the application is hosted. The container encapsulates all required dependencies.

Application Layer
This layer contains the React application responsible for rendering the user interface and displaying movie information.

This layered design improves system maintainability and ensures that each layer has a clearly defined role.

The architecture of the system is shown in **Figure 1**.

2.2 Process Flow

The process flow describes how the system operates when a user accesses the application.The sequence of operations is as follows:

1. The user opens the application URL in a web browser.
2. The browser sends an HTTP request to the Docker container.
3. The Docker container receives the request and processes it.
4. The React application serves static files including HTML, CSS, and JavaScript.
5. The browser renders the interface and displays the application to the user.

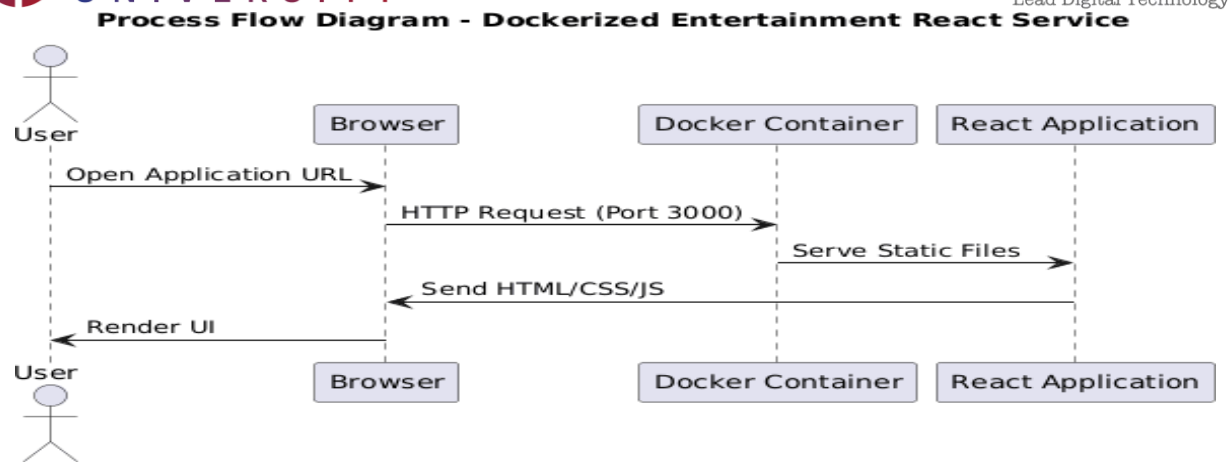

Figure 2: Process Flow Diagram

2.3 Information Flow

The information flow describes how data is transferred between different components of the system.

When a user interacts with the application, the browser sends requests to the Docker container. The container processes the request using the React application and returns the necessary user interface data.

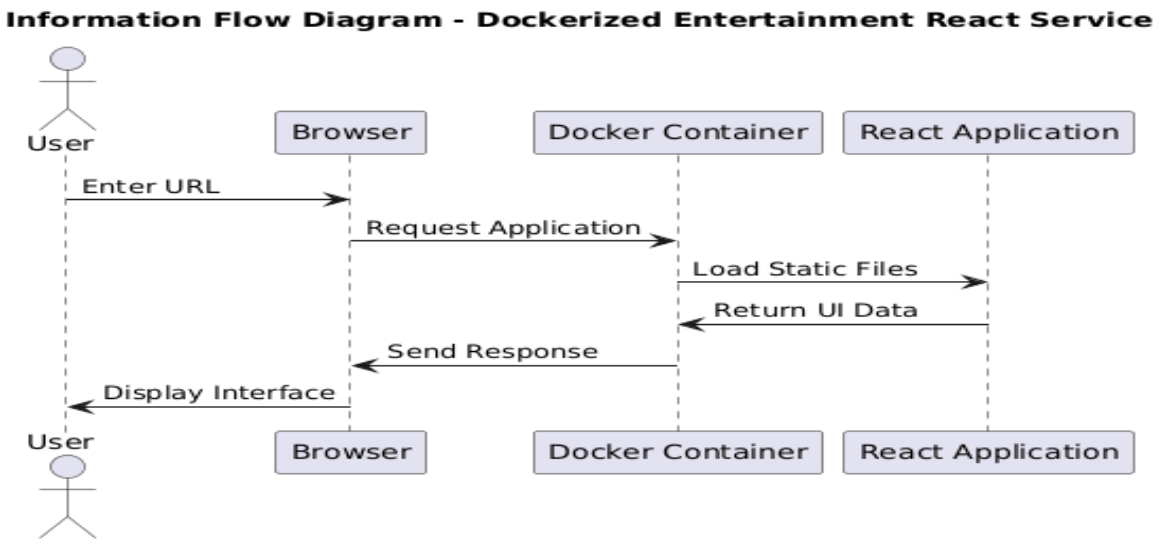

Figure 3: Information Flow Diagram

2.4 Components Design

The system consists of multiple components that work together to deliver the application.

React Frontend

The frontend application is developed using React. It is responsible for rendering the user interface and displaying movie-related information.

Dockerfile

The Dockerfile defines instructions required to build the Docker image. It specifies the base image, dependencies, and commands used to run the application.

Docker Image

The Docker image is a packaged version of the application including all required dependencies.

Docker Container

The Docker container is the runtime environment created from the Docker image. It executes the application and exposes the required port for browser access.

Browser

The browser acts as the client interface that sends requests to the system and displays the application interface.

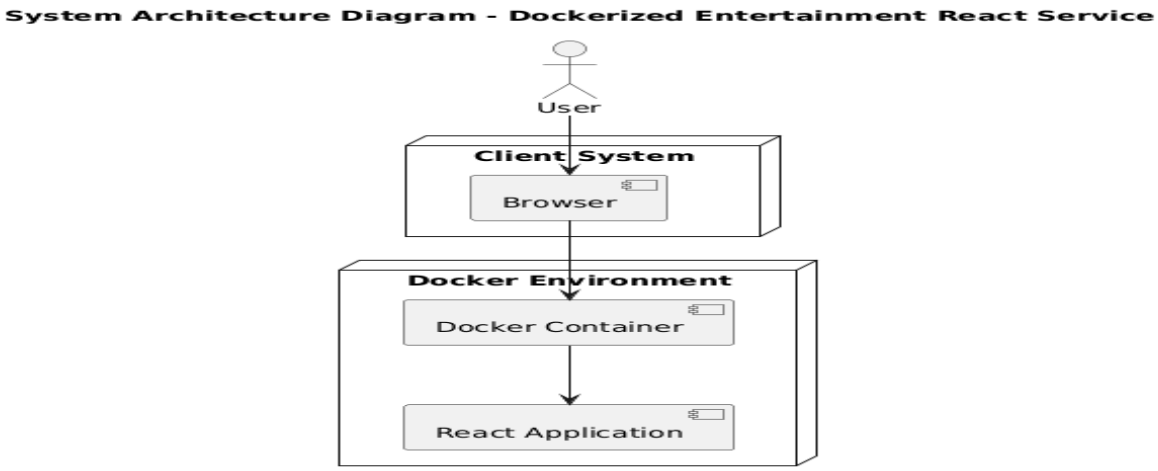

Figure 4: System Architecture Diagram

2.5 Key Design Considerations

Several design considerations were taken into account while developing the system.

Simplicity
The user interface is intentionally simple to focus on DevOps concepts rather than complex frontend development.

Portability
Docker containers ensure that the application behaves consistently across different systems.

Scalability
The system architecture allows future integration with Kubernetes for container orchestration and horizontal scaling.

Maintainability
The layered architecture makes the system easier to maintain and update.

Extensibility
Future enhancements such as backend APIs, database integration, and automated CI/CD pipelines can be easily added.

2.6 API Catalogue

The current implementation of the project focuses primarily on the frontend application. Therefore, no backend APIs are currently implemented.

However, the system architecture is designed in a way that backend APIs can be easily integrated in the future.

Example API endpoints that could be added include:

GET /movies
GET /movies/{id}
GET /movies/genre

These APIs would allow the system to retrieve movie data from a backend service or database.

3. Data Design

3.1 Data Model

The system manages movie-related information using a simple data model.

The main entity in the system is the **Movie** entity.

Attributes include:

Movie ID
Title
Genre
Rating
Release Year

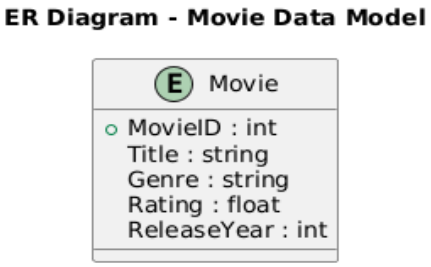

Figure 5: ER Diagram

3.2 Data Access Mechanism

Data within the system is accessed through the frontend application. In future versions, the system can be extended to include backend APIs that retrieve movie information from a database.

The React application would communicate with the backend service using REST API calls.

3.3 Data Retention Policies

The system stores only lightweight entertainment data and therefore does not require complex retention policies.

Application logs and temporary container files may be periodically cleaned to ensure efficient resource usage.

3.4 Data Migration

Since the current implementation does not include a database, data migration is not required.

However, if a database is introduced in the future, migration scripts may be used to transfer data between system versions.

4. Interfaces

The system provides several interfaces that allow interaction between different components.

User Interface:The user interface is developed using React and is accessed through a web browser.

Container Interface:Docker provides the runtime interface for executing the application.

Cloud Infrastructure Interface The application can be deployed on cloud platforms such as AWS.

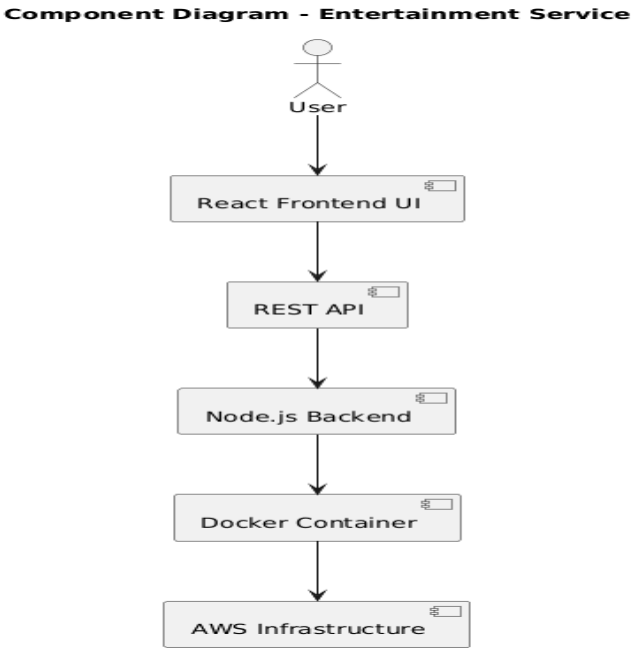

Figure 6: UML Component Diagram

5. State and Session Management

The application follows a **stateless architecture** where each request is processed independently.

Since the application currently serves static content, no session management mechanism is required. Future implementations may include authentication systems using token-based mechanisms.

6. Caching

Caching mechanisms improve application performance by reducing repeated data requests.

Possible caching techniques include:

Browser caching
Static file caching
Redis-based caching for backend services

7. Non-Functional Requirements

7.1 Security Aspects

Security considerations include:

Container isolation through Docker
Controlled deployment pipelines
Secure cloud infrastructure configurations

These measures ensure that the application runs securely within the containerized environment.

7.2 Performance Aspects

Performance optimization techniques include:

Static React builds
Efficient container execution
Lightweight frontend design

These strategies ensure fast loading times and efficient resource utilization.

8. References

Docker Official Documentation
React.js Documentation
Vite Documentation
DevOps Best Practices